

神经辅助量子路由编译器项目报告

算法设计与智能计算课程项目

目录

1 摘要	1
2 引言	1
3 问题建模	2
3.1 硬件拓扑	2
3.2 逻辑电路与映射	2
3.3 搜索状态与优化目标	3
4 算法设计	3
4.1 图预处理与前沿门	4
4.2 初始映射选择	4
4.3 神经网络动作评分	5
4.4 有界束搜索	5
4.5 触发剪枝与局部修复	5
4.6 轨迹复放验证	6
5 系统实现	6
5.1 接口层	7
5.2 编译层	7
5.3 模型与证据层	8
6 实验与分析	8
6.1 基线与指标	8
6.2 10/20 比特代表电路结果	9
6.3 30/50 比特扩展测试	10
6.4 综合分析	11
7 复杂度与课程算法思想	11
7.1 课程算法思想映射	12
7.2 SABRE 的基线意义	13
8 总结与局限	13
9 参考资料	14

A 附录：复现实验命令	15
A.1 环境检查	15
A.2 快速基线矩阵	15
A.3 公开项目检查	15

1 摘要

真实量子芯片通常只允许相邻物理量子比特直接执行双比特门，而逻辑量子电路往往假设任意两个量子比特都可以交互。量子路由编译需要在保持电路语义等价的前提下，选择逻辑到物理量子比特的映射，并插入必要的 SWAP 门，使电路满足目标硬件拓扑。这一任务本质上是图约束下的组合优化问题：映射选择、局部交换、前沿门推进和搜索剪枝会共同影响最终 SWAP 数、电路深度和编译耗时。

本项目实现了神经辅助量子路由流程 NPQR。NPQR 没有把神经网络作为直接生成完整路由的黑盒，而是把神经动作评分嵌入可解释的经典搜索框架。系统先生成并排序初始映射候选，再通过有界束搜索展开 SWAP 序列，并结合前沿触发、动作剪枝和局部后缀修复提升完成率。最终结果必须经过轨迹复放验证，确认每一步门和 SWAP 都满足硬件连接约束。SABRE basic 在本项目中只作为主要质量基线，不作为 NPQR 的隐藏回退路径。

最终评测给出三个层次的结论。第一，在代表性 10/20 比特电路上，NPQR 完成 10/10 个样例，并且 10/10 的 SWAP 数低于 SABRE basic；所有完成结果均通过轨迹复放，且没有使用 SABRE 回退路径。第二，在 30/50 比特有界扩展测试中，NPQR 完成 4/4 个已声明样例，并在这些样例上继续低于 SABRE basic，说明该流程不只适用于小规模演示。第三，80/100 比特实验用于界定边界：当前 CPU 有界运行尚不能把这些规模作为稳定完成能力声明。因此，本文的主张是：NPQR 已形成一个可运行、可验证、可解释的神经辅助路由系统，在课程代表场景中稳定优于 SABRE basic，并展示出到 50 比特选定结构的扩展潜力；本文不声称其全面超过 SABRE lookahead，也不声称其是最先进通用量子路由器。

2 引言

量子计算程序在逻辑层面描述量子门序列，但真实量子芯片在物理层面存在连接约束。例如，一个双比特门只能在相邻物理量子比特之间直接执行。当逻辑电路中的两个量子比特被映射到不相邻的物理位置时，编译器需要插入 SWAP 门来移动量子态。这些额外 SWAP 会增加门数和电路深度，进而影响噪声环境下的实际执行质量。

因此，量子路由是量子编译中的关键步骤。它的输入是量子电路和硬件拓扑，输出是满足拓扑约束的等价电路。该问题既包含图上的最短距离、映射选择和路径规划，也包含启发式搜索、局部改进和近似求解。若尝试穷举所有映射和 SWAP 序列，搜索空间会随量子比特数和双比特门数量快速增长，难以在课程项目或交互式系统中直接使用。

本项目的目标是实现一个神经辅助量子路由编译器。项目没有把神经网络设计成直接生成完整路由电路的黑盒模型，而是把神经网络放入经典搜索框架中，让模型在每个状态下评估候选 SWAP 动作。这样既能体现智能计算方法，也能保留图搜索和轨迹验证带来的可解释性。完整流程还引入初始映射选择、束搜索、触发剪枝和局部修复，使

系统能够在有限时间内寻找可行且质量较高的路线。

本文围绕以下内容展开：首先给出量子路由问题的图模型和优化目标；随后介绍 NPQR 算法的各个组成部分；接着说明系统如何通过多种接口提供服务；最后基于已有证据给出实验分析、复杂度分析、课程算法思想映射和项目局限。

3 问题建模

本节把量子路由问题形式化为图约束下的映射与搜索问题。该建模把量子硬件拓扑、逻辑电路和路由动作放到统一框架中，便于后续分析算法复杂度和设计启发式策略。

3.1 硬件拓扑

目标量子芯片表示为无向图

$$G = (V, E),$$

其中 V 表示物理量子比特集合， E 表示芯片上可直接执行双比特门的物理连接。若 $(u, v) \in E$ ，则映射到 u 和 v 上的两个逻辑量子比特可以直接执行双比特门；否则必须通过若干次 SWAP 改变映射后才能执行。

项目默认采用 IBM Tokyo 20Q 风格拓扑。系统会预先计算物理图中任意两点的最短路径距离 $d(u, v)$ 。该距离用于估计当前映射下前沿双比特门的执行难度，也用于启发式评分和动作筛选。

3.2 逻辑电路与映射

输入量子电路可以表示为门序列

$$C = (g_1, g_2, \dots, g_m),$$

其中一部分门是单比特门，另一部分门是双比特门。单比特门不受硬件连接约束；双比特门必须在相邻物理量子比特上执行。令逻辑量子比特集合为 Q ，当前映射为

$$\pi : Q \rightarrow V.$$

当双比特门 $g = (q_i, q_j)$ 满足 $(\pi(q_i), \pi(q_j)) \in E$ 时，该门在当前映射下可执行。否则，路由器需要选择一条物理边 (u, v) 插入 SWAP，并更新映射 π 。

一次 SWAP 可以看成对当前映射的局部置换。若物理边 (u, v) 上分别放置逻辑量子比特 q_a 和 q_b ，执行 SWAP 后映射会变为 $\pi'(q_a) = v$ 、 $\pi'(q_b) = u$ ，其他逻辑量子比特位置保持不变。由于每次 SWAP 都会改变后续所有双比特门的物理位置，路由问题不能简单拆成互不相关的最短路问题，而必须在连续状态空间中进行搜索。

3.3 搜索状态与优化目标

一个路由状态可以由三部分描述：当前映射 π 、已经执行的门前缀和待执行门集合。没有未满足依赖的待执行双比特门称为前沿门。每一步搜索要么执行当前可执行门，要么插入一个合法 SWAP 来改变映射。

路由目标是在保证电路语义等价和拓扑合法的前提下优化以下指标：

$$\min \alpha \cdot N_{\text{swap}} + \beta \cdot D + \gamma \cdot T,$$

其中 N_{swap} 是插入的 SWAP 数， D 是路由后电路深度， T 是编译耗时。在交互式演示和在线调用场景中，算法不能只追求理论最优，还必须保证运行时间可控。因此本项目采用近似搜索策略，在有限时间内寻找可验证的高质量可行解。

这里的目标函数用于说明多目标权衡，并不表示系统会精确求解一个全局最优整数规划。实际实现中，SWAP 数和深度反映路由质量，耗时反映在线调用可用性。三者之间存在天然冲突：更大的搜索宽度可能减少 SWAP，但会增加运行时间；更强的剪枝可以减少耗时，但可能错过更好的路线。

4 算法设计

NPQR 是一个神经辅助的组合式量子路由算法。它的核心不是单独的神经网络模型，而是“神经网络评分 + 经典搜索控制”的整体流程。图 1 展示了主要步骤，算法 2 给出高层伪代码。

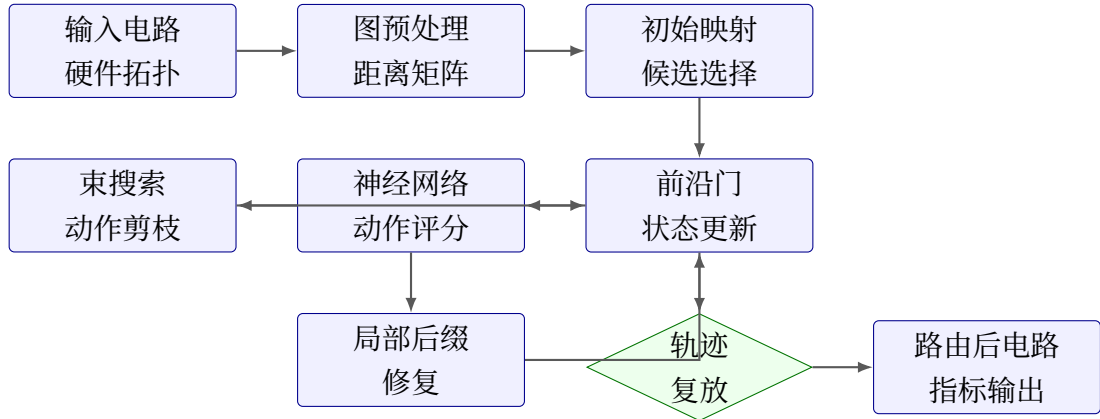


图 1: NPQR 神经辅助量子路由流程

从职责划分看，NPQR 的每个模块都解决搜索中的一个具体问题。表 1 概括了这些模块的输入、输出和作用。

表 1: NPQR 组件职责

组件	主要输入	主要输出	作用
图预处理	电路、硬件拓扑	依赖关系、距离矩阵	建立搜索所需的图结构和距离特征。
初始映射选择	逻辑交互、物理连接	候选初始映射	给搜索提供更好的起点，减少后续 SWAP 压力。
神经动作评分	当前状态、候选 SWAP	动作偏好分数	为搜索提供学习得到的局部判断。
有界束搜索	候选状态、动作分数	多条候选路线	降低一步贪心错误造成的失败风险。
前沿触发剪枝	前沿门、候选动作	裁剪后的动作集合	在困难状态增强搜索，在普通状态控制开销。
后缀修复	接近完成的路线	修复后的局部后缀	处理末端少量门卡住的情况。
轨迹复放	输出轨迹、硬件拓扑	合法性验证结果	确认最终路线满足拓扑约束并执行完原始门。

4.1 图预处理与前沿门

系统首先解析输入电路，提取门序列、量子比特集合和双比特门依赖关系。随后将目标芯片表示为无向图，并计算物理量子比特之间的最短路径距离矩阵。该预处理把后续多次距离查询变成常数时间访问，是典型的时空权衡。

在路由过程中，系统维护当前前沿门集合。若前沿双比特门在当前映射下已经相邻，系统直接执行该门并推进门依赖；若不能执行，系统生成候选 SWAP 动作，并进入评分与搜索流程。

4.2 初始映射选择

初始映射决定逻辑量子比特最开始放置在哪些物理量子比特上。若初始映射与电路的交互结构匹配，后续需要的 SWAP 会减少。NPQR 会根据逻辑交互关系和物理拓扑生成多个候选映射，并对候选进行排序。该步骤把原始电路路由问题转化为较好的搜索起点，降低后续局部搜索的压力。

4.3 神经网络动作评分

在每个非终止状态下，神经网络根据当前映射、前沿门、物理距离和候选 SWAP 动作输出动作偏好分数。该分数不直接决定完整答案，而是作为束搜索中的一个启发式信号。这种设计使模型能够参与决策，同时避免模型一次性生成完整电路带来的不可控风险。

为了便于解释，可以把一个候选动作 a 在状态 s 下的综合评分写成

$$F(s, a) = \lambda_1 P_\theta(a | s) - \lambda_2 H_{\text{frontier}}(s, a) - \lambda_3 C_{\text{depth}}(s, a) - \lambda_4 C_{\text{repeat}}(s, a),$$

其中 $P_\theta(a | s)$ 表示神经网络给出的动作偏好, H_{frontier} 表示前沿门距离启发式代价, C_{depth} 表示对深度增长的惩罚, C_{repeat} 表示对重复无效移动的惩罚。实际系统可以使用等价的排序逻辑；该公式强调的是：神经网络提供主要偏好，但最终决策仍受拓扑距离和搜索状态共同约束。

4.4 有界束搜索

单步贪心容易因为局部最优而卡住。NPQR 使用有界束搜索保留多条候选路线。每次展开时，系统对候选状态进行评分，只保留排名靠前的若干条路线继续搜索。束宽越大，搜索质量通常越高，但时间和空间消耗也随之增加。

束搜索的关键在于把“当前最优动作”扩展为“当前较优路线集合”。设第 t 层保留的状态集合为 $\mathcal{B}_t$ ，系统会对每个 $s \in \mathcal{B}_t$ 展开若干动作并生成候选集合 $\mathcal{C}_{t+1}$ ，然后按累计评分取前 B 个状态：

$$\mathcal{B}_{t+1} = \text{TopB}(\mathcal{C}_{t+1}).$$

这样做牺牲了一部分运行时间，但能减少单步评分误差造成的不可逆失败。

4.5 触发剪枝与局部修复

对普通样例，过重搜索会增加不必要耗时；对交互密集或随机性强的样例，过轻搜索又可能漏掉可行路线。NPQR 使用前沿触发规则识别更难的局部结构，只在必要时启用更强的前沿搜索。同时，动作剪枝会删除明显较差的 SWAP 候选，控制搜索树分支数。

触发规则主要关注前沿门的拥塞程度、候选动作是否能接触关键前沿量子比特、以及普通展开是否出现停滞。若状态表现为“前沿门距离长期不能下降”或“候选动作虽然得分相近但影响范围不同”，系统会扩大对前沿相关动作的考虑范围。动作剪枝则保留与当前前沿门、短路径和神经评分更相关的候选，避免在所有硬件边上无差别展开。

当一条路线已经完成大部分门但在末端卡住时，NPQR 会启用有界后缀修复。后缀修复不是全局穷举，而是在有限深度内围绕剩余门进行局部搜索，用于补齐接近完成的

路线。

4.6 轨迹复放验证

最终结果必须通过轨迹复放。系统会从初始映射开始重新模拟所有门和 SWAP，检查每一步是否满足硬件边约束，并确认所有原始门都已执行。只有通过复放验证的路线才会作为 NPQR 结果返回。该步骤把搜索输出转化为可审计的工程结果。

轨迹复放还用于区分“搜索过程看似完成”和“输出结果确实可执行”。如果某一步双比特门不在硬件边上，或某个原始门没有被执行，复放都会失败。这个设计对课程项目很重要：它把算法输出和硬件约束重新绑定，避免只报告一个未经验证的启发式分数。

算法 1: NPQR 神经辅助量子路由

输入: 量子电路 C ，硬件图 G ，束宽 B ，修复深度 R

输出: 路由后电路、SWAP 数、轨迹或失败标记

1. 解析 C ，构建门依赖和前沿门集合；
2. 计算 G 上的距离矩阵，生成初始映射候选；
3. 将候选映射转化为初始搜索状态集合；
4. **while** 存在未完成状态 **do**
 - 4.1 执行所有当前可直接执行的前沿门；
 - 4.2 若状态完成，进行轨迹复放并返回结果；
 - 4.3 生成合法 SWAP 候选动作；
 - 4.4 用神经网络和距离启发式为动作评分；
 - 4.5 根据触发规则决定是否扩大前沿搜索；
 - 4.6 剪枝弱动作，展开候选状态；
 - 4.7 保留评分最高的 B 个状态；
 - 4.8 若状态接近完成但卡住，尝试深度不超过 R 的后缀修复；
5. **return** 失败标记。

图 2: NPQR 高层伪代码

5 系统实现

项目不仅实现了算法流程，还提供了面向使用者的多种调用方式。系统调用链路如图 3 所示。网页、REST API、MCP 服务和命令行都可以触发同一套编译流程，并返回路由后电路、SWAP 数、深度、耗时、轨迹和 SABRE 基线指标。

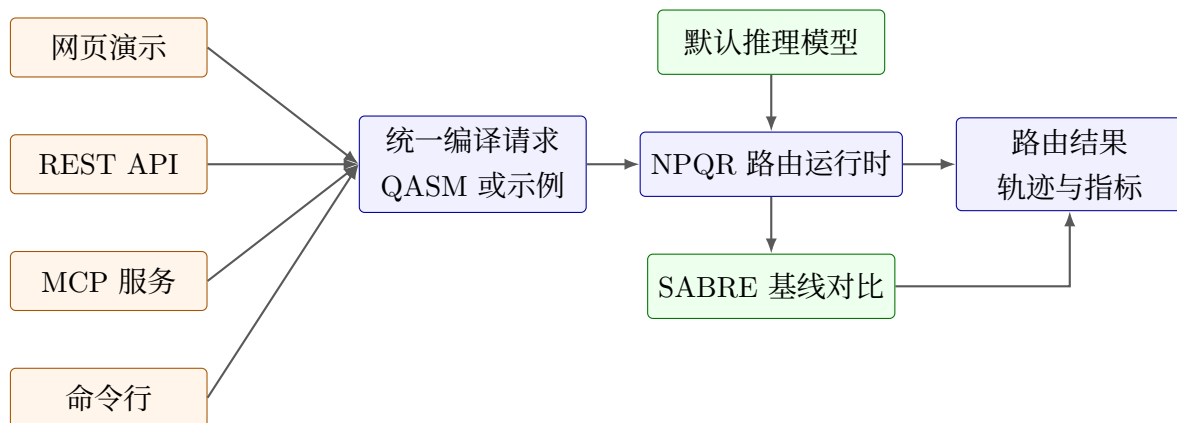


图 3: 系统调用链路

5.1 接口层

REST API 适合网页和普通后端调用。用户可以提交内置示例名称或自定义 OpenQASM 2 文本，系统返回结构化 JSON 结果。MCP 服务适合工具客户端和自动化场景，能够查询状态、列出示例、运行 NPQR、运行 SABRE 基线并返回算法证据。命令行适合本地检查、演示和快速基线矩阵生成。

这些入口的共同点是都把 NPQR 作为默认编译流程，同时把 SABRE 作为对比结果返回。接口中的对比字段用于解释基线表现，不表示 NPQR 失败时会由 SABRE 替代完成。

表 2: 接口返回信息及用途

返回信息	用途
路由后 QASM	作为可提交给后续量子工具链的编译结果。
SWAP 数	衡量由于硬件拓扑限制额外插入的交换门数量。
电路深度	衡量路由后电路执行层数，间接反映噪声暴露时间。
运行时间	衡量在线调用或课堂演示时的响应成本。
路由轨迹	展示每一步门执行和 SWAP 插入，便于复放和讲解算法。
SABRE 指标	作为固定基线对比，帮助解释结果质量边界。
完成状态	标记当前算法是否完成全部门并通过基本约束检查。

5.2 编译层

编译层负责把输入电路转化为路由任务，调用 NPQR 运行时，并将结果重新组织为用户可读的输出。输出包括路由后 QASM、插入 SWAP 数、电路深度、运行时间和

可复放轨迹。轨迹字段使用户能够观察每一步 SWAP 和门执行，也便于报告中解释算法过程。

系统实现中有两条质量控制线。第一条是输入约束：只接受限定长度内的 OpenQASM 2 文本，并使用固定硬件拓扑进行演示，避免在线调用时出现不可控输入规模。第二条是输出约束：返回结果必须带有完成状态、路由指标和轨迹信息。这样即使某个样例没有得到理想指标，调用者也能判断结果是否完成、是否可复放、与基线差距在哪里。

5.3 模型与证据层

项目包含一个默认推理模型。该模型在搜索循环中提供动作偏好，不单独承担完整路由生成。系统还提供机器可读的算法证据，说明默认路由、算法组成、SABRE basic 基线、10/20 比特代表电路结果、30/50 比特扩展测试和能力边界。该证据用于保证报告、网页和接口描述保持一致。

这种“模型 + 搜索 + 证据”的结构也降低了展示风险。报告可以明确说明神经网络在动作评分环节发挥作用，同时承认最终路由质量还依赖搜索控制和修复机制。对在线使用者而言，接口返回的是完整编译结果；对课程报告而言，证据层给出算法边界，避免把单个样例结果扩大为未经验证的总体结论。

6 实验与分析

本节给出项目最终采用的证据链。评测目标不是把 NPQR 描述成所有量子电路上的最优路由器，而是回答三个更具体的问题：第一，系统能否在代表性课程样例上稳定完成路由；第二，在完成且可复放的前提下，NPQR 的 SWAP 数是否低于 SABRE basic；第三，当问题规模扩大到 30/50/80/100 比特时，当前系统的可靠能力边界在哪里。这样的评测口径能避免把单个成功样例扩大为通用结论，也能清楚说明算法的真实价值。

6.1 基线与指标

SABRE 是量子路由领域常用的传统启发式算法。项目使用 SABRE basic 作为主要对比基线，因为它稳定、易复现，并能代表一个工程上常见的传统路由器。SABRE 在本报告中只承担基线角色：它用于生成对照指标，不参与 NPQR 的路线生成，也不在 NPQR 失败时作为回退路径返回成功结果。

实验分析使用的主要指标如表 3 所示。完成状态和轨迹复放是底线指标；一个结果只有在所有原始门都被执行、所有双比特门都满足硬件边约束、且复放通过时，才被视为有效完成。在此基础上，SWAP 数用于衡量路由质量，电路深度和耗时用于衡量工程代价。由于 NPQR 包含神经推理和束搜索，本文不把运行时间超过 SABRE 视为失败；时间结果主要用于界定可用范围和后续优化方向。

表 3: 实验指标说明

指标	含义	解释方式
完成状态	是否成功生成满足约束的路由结果	是底线指标，未完成时 SWAP 和深度比较意义有限。
SWAP 数	插入的交换门数量	越低通常表示路由额外开销越小。
电路深度	路由后门序列的层数	越低通常表示潜在执行时间和噪声暴露越少。
运行时间	编译请求耗时	越低越适合在线接口和课堂演示。
轨迹复放	按输出轨迹重新验证拓扑合法性	用于确认结果不是仅由启发式分数推断。
基线差异	与 SABRE 指标的对照	用于观察算法表现边界，不作为唯一成功标准。

6.2 10/20 比特代表电路结果

表 4 汇总了最终用于课程报告的 10/20 比特代表电路结果。这组样例包含链式纠缠、傅里叶变换、QAOA、VQE 风格线路、随机线路，以及多种 20 比特用户自设计结构。它覆盖了结构化电路和较复杂交互电路两类场景，是本项目的主要性能证据。所有 NPQR 完成结果均不使用 SABRE 回退路径，并通过轨迹复放验证。

表 4: 10/20 比特代表电路与 SABRE basic 的最终对比

电路	比特数	NPQR SWAP	SABRE basic SWAP	差值
GHZ10	10	2	6	-4
QFT10	10	6	13	-7
QAOA10	10	0	15	-15
VQE10	10	1	4	-3
Random10	10	17	19	-2
Brickwork20	20	15	31	-16
CliqueBlocks20	20	19	38	-19
DeepRandom20	20	105	135	-30
RingEntangler20	20	16	41	-25
SparseRandom20	20	37	57	-20

从表中可以看到，NPQR 在 10 个代表样例上全部完成路由，并且每个样例的 SWAP

数都低于 SABRE basic。差值最大的样例包括 DeepRandom20、RingEntangler20 和 SparseRandom20，说明神经评分、初始映射和前沿搜索组合在较复杂的 20 比特交互结构上具有实际收益。QAOA10 的 NPQR 结果不需要额外 SWAP，体现了结构化线路中初始映射选择的作用。Random10 的优势相对较小，这也说明 NPQR 的收益不是来自统一地压低所有数字，而是依赖电路结构和搜索策略是否匹配。

6.3 30/50 比特扩展测试

为了检查系统是否只适用于 20 比特以内的小样例，项目还进行了 30/50 比特扩展测试。该实验不是全面大规模基准，而是一个有界能力测试：它关注当前流程能否在更大拓扑上完成选定样例，且完成结果是否仍具有质量优势。目标拓扑使用 30 比特和 50 比特网格，仍以 SABRE basic 作为对比。

表 5: 30/50 比特扩展测试与 SABRE basic 的对比

电路	比特数	NPQR SWAP	SABRE basic SWAP	差值
LineGHZ30	30	23	43	-20
Random30-d4	30	61	67	-6
LineGHZ50	50	50	79	-29
RingSparse50	50	98	113	-15

结果显示，NPQR 可以在已声明的 30/50 比特样例上完成路由。在 LineGHZ30、Random30-d4、LineGHZ50 和 RingSparse50 上，NPQR 的 SWAP 数均低于 SABRE basic。这说明当前流程已经具备超过 20 比特演示规模的有效工作区间，最大已证明完成并优于 SABRE basic 的规模达到 50 比特。需要强调的是，这仍然是选定样例测试，不能表述为所有 50 比特电路或所有大规模电路都全面优于 SABRE。

表 6: 80/100 比特有界边界测试

电路	比特数	NPQR SWAP	SABRE basic SWAP	结论
LineGHZ80	80	—	137	240 秒 CPU 有界运行未完成
Random80-d2	80	—	—	外层有界运行停止前未完成
LineGHZ100	100	—	—	未作为最终能力声明
RingSparse100	100	—	—	未作为最终能力声明

进一步的 80/100 比特探索用于界定当前上限，而不是作为成功声明。LineGHZ80 在 240 秒 CPU 有界运行内未完成；Random80-d2 在外层有界运行停止前未完成；100 比特样例未继续作为最终能力声明。因此，结项报告中最稳妥、也最有证据支撑的能力范围是：10/20 比特为主性能结论，30/50 比特为已完成的扩展证据，80/100 比特为当前边界。

6.4 综合分析

最终实验支持以下结论：NPQR 是一条可运行的神经辅助量子路由流程；它在 10/20 比特代表电路上相对 SABRE basic 取得一致 SWAP 优势；它的结果经过轨迹复放验证，且 SABRE 只作为对比基线，不参与 NPQR 路线生成。扩展测试还表明，该流程可以运行到 30/50 比特，并在所测结构上保持优势。

从算法角度看，这些结果说明本项目的有效性主要来自组合设计，而不是单一模型能力。初始映射减少了搜索起点错误，神经评分提供了动作偏好，有界束搜索避免了过早贪心，前沿触发和动作剪枝控制了复杂样例的候选展开，后缀修复补齐了接近完成的路线，轨迹复放则保证了结果可审计。若只保留其中任一部分，系统很难同时做到完成、低 SWAP 和可验证。

实验也保留明确边界。首先，NPQR 的运行时间明显高于 SABRE，原因是它包含神经推理、候选映射、前沿搜索和复放验证。其次，NPQR 并不保证能在 80/100 比特规模完成路由，当前 CPU 有界测试已经观察到 80 比特 completion 边界。再次，SABRE lookahead 不是本文的主比较目标，部分样例上 lookahead 仍可能取得更低 SWAP。综合来看，课程项目的合理结论是：NPQR 在代表性 10/20 比特场景中实现了对 SABRE basic 的稳定质量优势，并把已证明扩展能力推进到选定 50 比特结构；50 比特以上的任务仍需要进一步的初始映射、候选剪枝和分层搜索优化。

7 复杂度与课程算法思想

NPQR 的复杂度主要来自候选映射、候选 SWAP 展开、束搜索和局部修复。令 $n = |V|$ 表示物理量子比特数， m 表示电路门数， $e = |E|$ 表示硬件边数， K 表示初始映射候选数， B 表示束宽， D 表示主搜索最大展开深度， A 表示每个状态保留的候选动作数， R 表示后缀修复深度。

表 7: NPQR 主要步骤复杂度

步骤	时间复杂度估计	说明
拓扑距离预处理	$O(n^3)$ 或 $O(n(e + n \log n))$	取决于使用 Floyd 或多源最短路径; Tokyo 规模较小, 开销可控。
电路依赖处理	$O(m)$	扫描门序列, 维护前沿门和依赖状态。
初始映射候选	$O(K \cdot n^2)$ 量级	与候选数和交互评分方式有关。
主束搜索	$O(D \cdot B \cdot A \cdot C_s)$	C_s 为单次状态评分和更新成本。
后缀修复	$O(A^R)$ 的受限搜索	通过小深度 R 限制局部搜索规模。
轨迹复放	$O(m + N_{\text{swap}})$	线性验证输出轨迹是否合法。

复杂度表明, 若不加剪枝, 搜索空间会随深度指数增长。NPQR 通过固定束宽、动作剪枝和有界修复把搜索规模限制在可控范围内。因此, 该算法属于近似搜索框架: 它追求有限时间内的高质量可行解, 而不保证全局最优。

从参数影响看, B 和 A 是主搜索成本的主要控制项。增大 B 会保留更多候选路线, 有利于降低局部最优风险; 增大 A 会让每个状态尝试更多 SWAP, 可能提高完成率。但两者都会线性放大主搜索开销。修复深度 R 的影响更强, 因为局部修复最坏情况下呈指数增长。因此, 项目把修复限定在接近完成的状态上, 而不是在整个搜索过程中频繁启动。

7.1 课程算法思想映射

项目可以对应到多种课程算法设计思想。表 8 给出主要对应关系。这种映射不是把每个经典算法完整复刻一遍, 而是说明 NPQR 在问题建模和工程实现中如何使用这些思想。

表 8: 课程算法思想与项目对应关系

课程概念	在项目中的体现
图问题	芯片拓扑是图，量子比特是顶点，硬件连接是边。
变治法	把电路路由转化为拓扑图上的映射和路径规划问题。
贪婪启发式	使用前沿门距离下降等局部指标评价 SWAP 动作。
减治法	每执行一个合法门，剩余待路由任务减少。
时空权衡	预计算距离矩阵并保留候选路线，以空间换取重复查询效率和搜索质量。
迭代改进	路线通过连续 SWAP、映射更新和后缀修复逐步改善。
搜索剪枝	束宽、触发规则和动作剪枝限制搜索树规模。
近似算法	在有限时间内寻找高质量可行解，不保证全局最优。
神经网络	模型为候选 SWAP 动作提供偏好评分。

7.2 SABRE 的基线意义

SABRE 的核心思想可以看作贪婪启发式与有限前瞻的结合。它根据前沿门和后续门在物理拓扑上的距离选择 SWAP，通常能以较低时间成本给出稳定路线。NPQR 与 SABRE 的区别在于：NPQR 引入神经网络动作评分，并在搜索中保留多条候选路线，同时加入局部修复和轨迹复放。两者可以用于对比，但不能把 SABRE 结果作为 NPQR 的成功结果。

这一区别也对应课程中的“启发式搜索”与“智能计算”结合。SABRE 更接近人工设计的启发式函数，NPQR 则把启发式函数的一部分交给模型学习，再用显式搜索保证约束和可解释性。二者的比较能说明：神经网络并不自动解决组合优化问题，合理的状态表示、搜索边界和验证机制同样关键。

8 总结与局限

本项目完成了一条面向受限量子硬件的神经辅助量子路由流程。项目首先把量子路由建模为硬件耦合图上的映射与搜索问题，然后实现 NPQR：用神经网络评估候选 SWAP 动作，用初始映射选择提供较好的搜索起点，用有界束搜索保留多条候选路线，用前沿触发和动作剪枝控制搜索规模，用局部后缀修复处理接近完成但卡住的路线，并用轨迹复放验证最终电路。系统还提供命令行、REST API、MCP 服务和网页演示入口，能够返回路由后电路、SWAP 数、深度、耗时、路线轨迹和 SABRE basic 基线对比。

本项目的核心结论可以概括为三点。第一，NPQR 已经不是一个只停留在离线训练或单个脚本中的模型实验，而是一个可调用、可复验、可展示的完整路由系统。第二，在

最终代表性 10/20 比特评测中, NPQR 完成 10/10 个样例, 并且 10/10 的 SWAP 数低于 SABRE basic; 所有完成结果都不依赖 SABRE 回退路径, 并通过轨迹复放验证。第三, 30/50 比特扩展测试进一步证明当前流程具备一定扩展能力: 已声明的 4/4 个 30/50 比特样例全部完成, 并且全部优于 SABRE basic。基于这些证据, 项目可以合理宣称: 在课程代表场景和选定 50 比特结构上, NPQR 相对 SABRE basic 取得了可复验的质量优势。

这些结果的意义不只是少数表格指标的提升。更重要的是, 项目展示了一种适合组合优化问题的智能计算路线: 神经网络不直接替代搜索, 而是作为启发式评分函数嵌入可解释的搜索框架。这样既保留了模型对复杂状态的表达能力, 又保留了经典搜索在约束检查、局部修复和失败边界上的可控性。从课程角度看, 项目覆盖了图建模、变治法、贪婪启发式、减治法、时空权衡、迭代改进、搜索剪枝、近似算法和神经网络推理等内容, 是“组合优化 + 神经启发式 + 工程验证”的完整案例。

项目也有清晰边界。首先, NPQR 的运行时间明显高于 SABRE basic, 这是神经推理、候选映射、束搜索和轨迹复放带来的工程成本; 本项目以完成率和 SWAP 质量为主要指标, 不把速度超过 SABRE 作为结项目标。其次, NPQR 不应被表述为通用最先进量子路由器, 也不保证在所有电路上优于 SABRE。SABRE lookahead 不是本文主比较目标, 部分样例上仍可能更强。再次, 80/100 比特实验已经暴露当前边界: LineGHZ80 在 240 秒 CPU 有界运行内未完成, 100 比特样例没有作为最终能力声明。因此, 当前已证明的稳定上界应写作选定 50 比特结构, 而不是任意 50 比特以上电路。

如果未来继续推进, 最优方向不是盲目增加训练时间或直接启动 GPU, 而是围绕大规模路由瓶颈做结构性优化。第一, 改进 topology-aware initial mapping, 让大规模电路从更合理的物理布局开始。第二, 设计更强的 frontier candidate pruning, 减少 50 比特以上搜索中的无效候选展开。第三, 引入 profile selector, 根据电路结构自动选择轻量路由、前沿增强路由或分层搜索。第四, 在 80/100 比特规模上探索 hierarchical routing, 把全局路由拆成更小的拓扑区域再合并。只有当这些诊断证明瓶颈主要来自模型排序能力时, 才值得把 GPU 训练作为后续手段。

综合来看, 本项目已经达到课程作业的高质量结项标准: 它有明确的问题建模, 有可解释的算法流程, 有可运行的系统接口, 有不依赖回退路径的复放证据, 有相对 SABRE basic 的稳定 10/20 比特优势, 也有选定 30/50 比特扩展结果和诚实的 80/100 比特边界。最终结论不是夸大为“全面超过所有路由算法”, 而是更准确地说明: NPQR 在代表性中小规模量子路由任务上形成了质量优于 SABRE basic 的神经辅助搜索系统, 并展示了继续扩展到更大规模的可行方向。

9 参考资料

本报告参考了量子编译和神经辅助搜索方向的公开资料, 并结合项目已有接口和证据撰写。由于本文定位为课程项目报告, 参考资料以方法背景和实现依据为主。

1. G. Li, Y. Ding, and Y. Xie. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. *ASPLOS*, 2019. arXiv:1809.02573.
2. Qiskit Documentation. Transpiler and SabreSwap pass documentation.
3. A. Zulehner, A. Paler, and R. Wille. An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. *IEEE TCAD*, 2019.
4. A. Sinha, U. Azad, and H. Singh. Qubit Routing using Graph Neural Network aided Monte Carlo Tree Search. arXiv:2104.01992, 2021.
5. W. Tang, Y. Duan, Y. Kharkov, R. Fakoor, E. Kessler, and Y. Shi. AlphaRouter: Quantum Circuit Routing with Reinforcement Learning and Tree Search. arXiv:2410.05115, 2024.
6. 项目公开算法证据、接口说明和示例运行结果。

A 附录：复现实验命令

本附录列出报告中使用的公开检查命令，便于复现实验表格和验证接口状态。正文避免展开具体源码结构，本附录只保留必要命令。

A.1 环境检查

```
qcompiler info
```

A.2 快速基线矩阵

```
python scripts/experiment_algorithm_matrix.py --quick --json
```

A.3 公开项目检查

```
python scripts/check_submission_readiness.py
python -m pytest -q tests/test_public_api.py \
    tests/test_readme_contract.py \
    tests/test_submission_readiness.py
```